

Lab 2 - Core Terraform Advanced: Dynamic Blocks, Provisioners, Import and State Surgery

Full Lab Guide, Configuration & Source Code

Terraform / Ansible / Docker / Kubernetes Lab Series

Contents

Use Case	1
Prerequisites	2
Directory Layout	2
Part A — Dynamic Blocks & Provisioners	2
Step 1 — Configure variables	2
Step 2 — Plan and apply	2
Step 3 — Validate dynamic block output	2
Step 4 — Validate the provisioner ran	2
Part B — Import an Out-of-Band Resource	3
Step 5 — Simulate a “console-created” security group	3
Step 6 — Import it into the <code>aws_security_group.imported_sg</code> address	3
Step 7 — Reconcile config vs. real resource	3
Part C — State Surgery (<code>state mv</code> / <code>state rm</code>)	3
Step 8 — Rename a resource in state without destroying it	3
Step 9 — Remove a resource from state (stop managing, keep the real thing)	3
Validation Checklist	3
Cleanup	4
Key Concepts Reinforced	4
Appendix: Full Source Code	5
<code>main.tf</code>	5
<code>outputs.tf</code>	7
<code>variables.tf</code>	7

Use Case

A platform team inherited security groups created by hand in the AWS console and wants to (a) stop writing repetitive `ingress {}` blocks for every port rule, (b) run a one-time bootstrap command on new instances immediately at creation, and (c) bring the console-created security group under Terraform management **without recreating it** — plus learn how to safely rename/move resources in state without destroying real infrastructure.

This lab teaches: - **Dynamic blocks** to generate repeated nested blocks (`ingress`) from a list variable - **Provisioners**: `remote-exec` (runs on the new resource via SSH) and `local-exec` via `null_resource` (runs on the Terraform host) - **`terraform import`** to adopt existing, unmanaged infrastructure - **`terraform state mv` / `state rm`** for safe state surgery

Prerequisites

- Terraform $\geq$ 1.5.0, AWS CLI configured
- An existing VPC ID (`var.vpc_id`) — e.g. `module.vpc.vpc_id` from Lab 1, or your default VPC (`aws ec2 describe-vpcs`)
- An EC2 key pair + the matching `.pem` private key file locally

Directory Layout

```
lab2-core-dynamic-provisioners-import/  
├─ main.tf          # dynamic SG, provisioner-driven instance, import target  
├─ variables.tf     # ingress_rules list drives the dynamic block  
└─ outputs.tf
```

Part A — Dynamic Blocks & Provisioners

Step 1 — Configure variables

```
terraform init
```

Create `terraform.tfvars`:

```
vpc_id          = "vpc-xxxxxxx"  
key_name        = "my-ec2-keypair"  
private_key_path = "~/ssh/my-ec2-keypair.pem"
```

Step 2 — Plan and apply

```
terraform plan -out=lab2.plan  
terraform apply lab2.plan
```

Watch the apply logs — you'll see the `remote-exec` provisioner streaming `apt-get update`, `apt-get install nginx` output live, then the `null_resource.record_ip` `local-exec` writing `last_deployed_ip.txt`.

Step 3 — Validate dynamic block output

```
aws ec2 describe-security-groups --group-ids $(terraform output -raw dynamic_sg_id) \  
--query 'SecurityGroups[0].IpPermissions[0].{Port:FromPort,CIDR:IpRanges[0].CidrIp}'
```

You should see 3 rules (22, 80, 8080) even though the resource block only contains ONE dynamic "ingress" block — proof the loop expanded correctly. Add a 4th entry to `ingress_rules` in a `.tfvars` override and re-apply to confirm it updates without any code change.

Step 4 — Validate the provisioner ran

```
curl -s -o /dev/null -w "%{http_code}\n" http://$(terraform output -raw instance_public_ip)  
# Expect: 200  
cat last_deployed_ip.txt # should match instance_public_ip
```

Part B — Import an Out-of-Band Resource

Step 5 — Simulate a “console-created” security group

```
MANUAL_SG_ID=$(aws ec2 create-security-group \
  --group-name lab2-manual-sg \
  --description "created manually, will be imported" \
  --vpc-id <your-vpc-id> \
  --query 'GroupId' --output text)

aws ec2 authorize-security-group-ingress \
  --group-id $MANUAL_SG_ID --protocol tcp --port 443 --cidr 0.0.0.0/0
```

Step 6 — Import it into the `aws_security_group.imported_sg` address

```
terraform import aws_security_group.imported_sg $MANUAL_SG_ID
```

Step 7 — Reconcile config vs. real resource

```
terraform plan
```

Terraform will show a diff wherever `main.tf`’s `imported_sg` config doesn’t exactly match the real SG (e.g. `name_prefix` vs the fixed name AWS assigned). Adjust `main.tf` (e.g. use `name = "lab2-manual-sg"` instead of `name_prefix`) until `terraform plan` reports **no changes** — this proves the resource is now fully described by code, not just adopted blindly.

Part C — State Surgery (`state mv` / `state rm`)

Step 8 — Rename a resource in state without destroying it

Suppose you rename the resource block from `aws_security_group.imported_sg` to `aws_security_group.legacy_sg` in `main.tf`. Naively re-applying would destroy and recreate it. Instead:

```
terraform state mv aws_security_group.imported_sg aws_security_group.legacy_sg
terraform plan    # should show "No changes" if the rename in code matches
```

Step 9 — Remove a resource from state (stop managing, keep the real thing)

```
terraform state rm aws_security_group.legacy_sg
terraform plan    # Terraform now proposes to CREATE it again since it's unmanaged
```

This demonstrates `state rm` detaches Terraform’s bookkeeping while leaving the actual AWS resource untouched (verify with `aws ec2 describe-security-groups --group-ids $MANUAL_SG_ID` — it still exists).

Validation Checklist

Check	Command	Expected
Dynamic block rule count	<code>aws ec2 describe-security-groups ...</code>	3 (or N) ingress rules from 1 block
Provisioner success	<code>curl instance IP</code>	HTTP 200 from nginx

Check	Command	Expected
local-exec side effect	<code>cat last_deployed_ip.txt</code>	matches terraform output <code>instance_public_ip</code>
Import success	<code>terraform plan after import</code>	"No changes" once config matches
State rename safety	<code>terraform state mv then plan</code>	"No changes" (no destroy/create)
State rm behavior	<code>terraform state rm then plan</code>	Proposes create; real AWS resource still exists

Cleanup

```
terraform destroy
aws ec2 delete-security-group --group-id $MANUAL_SG_ID  # if still orphaned after state rm
rm -f last_deployed_ip.txt
```

Key Concepts Reinforced

- Dynamic blocks reduce repetition and make rule sets data-driven
- Provisioners are a last resort (prefer Ansible/cloud-init for real config management — see Labs 3–6) but are essential to understand for bootstrapping
- `import` + iterative `plan` is the safe way to adopt existing infrastructure
- `state mv`/`state rm` let you refactor code without destructive side effects

Appendix: Full Source Code

Every configuration and code file for this lab is listed below in full, organized by path, so the lab can be reproduced without needing the original repository.

main.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

# -----
# 1) DYNAMIC BLOCKS
# Instead of writing one ingress{} block per rule, we iterate over
# var.ingress_rules with a dynamic block. Add/remove entries in the
# variable and the SG updates without touching this resource code.
# -----
resource "aws_security_group" "dynamic_sg" {
  name_prefix = "lab2-dynamic-sg-"
  vpc_id      = var.vpc_id

  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      description = ingress.value.description
      from_port   = ingress.value.from_port
      to_port     = ingress.value.to_port
      protocol    = ingress.value.protocol
      cidr_blocks = ingress.value.cidr_blocks
    }
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "lab2-dynamic-sg" }
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
}
```

```

    filter {
      name     = "name"
      values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
    }
  }

data "aws_subnets" "default_vpc_subnets" {
  filter {
    name     = "vpc-id"
    values = [var.vpc_id]
  }
}

# -----
# 2) PROVISIONERS
#   remote-exec: runs commands ON the instance over SSH right after
#                 creation (installs nginx as a smoke test).
#   local-exec:  runs a command on the machine running Terraform
#                 (writes the instance IP to a local file) – driven
#                 by a null_resource so it can be re-triggered
#                 independently of the instance lifecycle.
# -----
resource "aws_instance" "demo" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
  subnet_id     = data.aws_subnets.default_vpc_subnets.ids[0]
  vpc_security_group_ids = [aws_security_group.dynamic_sg.id]
  key_name      = var.key_name

  tags = { Name = "lab2-provisioner-demo" }

  connection {
    type        = "ssh"
    user        = "ubuntu"
    private_key = file(var.private_key_path)
    host        = self.public_ip
    timeout     = "3m"
  }

  provisioner "remote-exec" {
    inline = [
      "sudo apt-get update -y",
      "sudo apt-get install -y nginx",
      "sudo systemctl enable nginx",
      "sudo systemctl start nginx"
    ]
  }
}

resource "null_resource" "record_ip" {
  triggers = {
    instance_id = aws_instance.demo.id
  }

  provisioner "local-exec" {

```

```

        command = "echo \"${aws_instance.demo.public_ip}\" > ${path.module}/last_deployed_ip.txt"
    }
}

# -----
# 3) IMPORT TARGET
# A second, manually-pre-existing-style SG resource block that we will
# NOT create with `apply` first – instead we create it out-of-band
# (see README "Import" section) and bring it under Terraform management
# with `terraform import`.
# -----
resource "aws_security_group" "imported_sg" {
    name_prefix = "lab2-imported-sg-"
    vpc_id      = var.vpc_id
    description = "Managed by Terraform after import"

    ingress {
        description = "HTTPS"
        from_port   = 443
        to_port     = 443
        protocol    = "tcp"
        cidr_blocks = ["0.0.0.0/0"]
    }

    egress {
        from_port   = 0
        to_port     = 0
        protocol    = "-1"
        cidr_blocks = ["0.0.0.0/0"]
    }

    lifecycle {
        create_before_destroy = true
    }
}

```

outputs.tf

```

output "instance_public_ip" {
    value = aws_instance.demo.public_ip
}

output "dynamic_sg_id" {
    value = aws_security_group.dynamic_sg.id
}

output "imported_sg_id" {
    value = aws_security_group.imported_sg.id
}

```

variables.tf

```

variable "aws_region" {
    default = "us-east-1"
}

```

```
variable "vpc_id" {
  description = "VPC to attach security group to"
  type        = string
}

variable "key_name" {
  type = string
}

variable "private_key_path" {
  description = "Path to the private key matching key_name, used for remote-exec provisioner"
  type        = string
}

# Dynamic block driver: arbitrary number of ingress rules defined purely in data,
# no repeated ingress{} blocks needed in the resource.
variable "ingress_rules" {
  type = list(object({
    description = string
    from_port   = number
    to_port     = number
    protocol    = string
    cidr_blocks = list(string)
  }))
  default = [
    {
      description = "SSH"
      from_port   = 22
      to_port     = 22
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    },
    {
      description = "HTTP"
      from_port   = 80
      to_port     = 80
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    },
    {
      description = "App Port"
      from_port   = 8080
      to_port     = 8080
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  ]
}
```